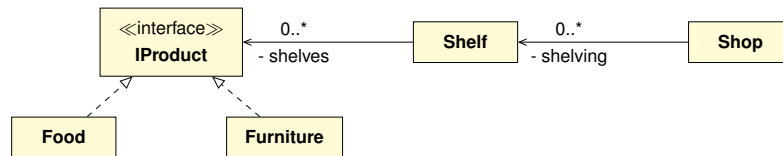


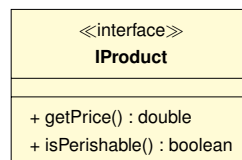
Exercice 1 : Mise en rayon

Nous cherchons à simuler une mise en rayon de différents produits, en sachant que certains produits sont périssables (mais pas tous) et que certains rayonnages sont réfrigérés (mais pas tous). Du coup, un produit ne peut pas être mis en rayon n'importe où : cela va dépendre de ses caractéristiques.

On ne souhaite pas traiter la mise en rayon différemment selon les types de produits. Au contraire, il est préférable d'uniformiser le traitement pour que cela soit transparent (sans faire de test pour déterminer son type réel). Voici le diagramme UML décrivant l'organisation générale à mettre en place :

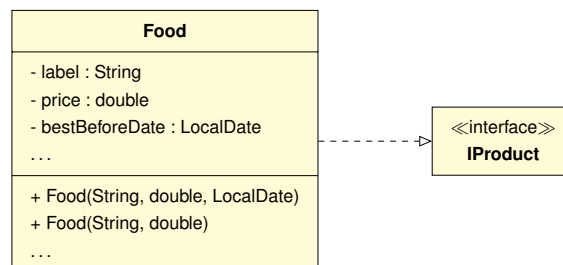


Q1. Écrivez l'interface `IProduct` définie de la manière suivante :



Q2. Intéressons-nous d'abord aux produits périssables.

Q2.1. Écrivez la classe `Food` représentant un produit périssable, et qui implémente l'interface `IProduct`.



De plus, lors de la création d'une instance, si le paramètre correspondant au `label` est `null`, il doit être remplacé par `"refUnknownXXX"` où `"XXX"` est la `XXX`-ème référence inconnue. Si la date limite n'est pas spécifiée, elle doit alors être fixée à dix jours de la date courante.

Q2.2. Munissez cette classe des accesseurs ainsi que de la méthode `isPerishable()`.

```
boolean isPerishable()
```

Q2.3. Ajoutez une méthode `toString` pour que la chaîne `[pasta=3.25 -> before 2019-01-01]` représente un paquet de pâtes coûtant 3.25 euros à consommer de préférence avant le 2019-01-01.

Q2.4. Écrivez une méthode `isBestBefore` déterminant si sa date limite est antérieure à une date donnée en paramètre.

```
boolean isBestBefore(LocalDate aDate)
```

Q3. Si on remplit une étagère avec de la nourriture, on souhaite pouvoir mettre les articles de date limite proche en premier. Pour cela, il faut pouvoir trier la nourriture par date limite. Modifiez votre code afin de permettre l'exécution du programme suivant :

```

public class UseComparable {
    public static void main(String[] args) {
        Food f1 = new Food("pasta", 3.25, LocalDate.of(2019, 1, 1));
        Food f2 = new Food("fish", 10.0, LocalDate.of(2019, 1, 10));
        Food f3 = new Food("meat", 15.0, LocalDate.of(2019, 1, 3));
        ArrayList<Food> storage = new ArrayList<Food>();
        storage.add(f1); storage.add(f2); storage.add(f3);
        System.out.println(storage);
        Collections.sort(storage);
        System.out.println(storage);
    }
}

```

On doit obtenir l'affichage suivant :

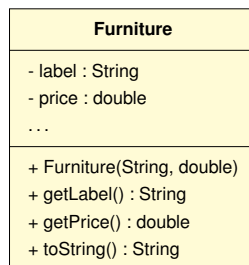
```

$> [[pasta=3.25 --> before 2019-01-01], [fish=10.0 --> before 2019-01-10], [meat=15.0 --> before
↪ 2019-01-03]]
$> [[pasta=3.25 --> before 2019-01-01], [meat=15.0 --> before 2019-01-03], [fish=10.0 --> before
↪ 2019-01-10]]

```

Q4. Concentrons-nous maintenant sur les produits non périssables.

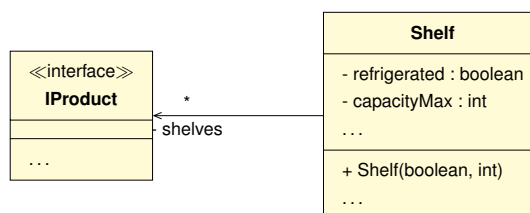
Q4.1. Créez une classe `Furniture`, implémentant l'interface `IProduct` et spécifiée de la manière suivante :



Notons que, comme pour la classe `Food`, si le label fournit est `null`, il doit être remplacé par `"refUnknownXXX"` où `"XXX"` indique la XXX-ème référence inconnue. La méthode `toString` doit avoir un comportement similaire à celui de la classe `Food`.

Q5. Intéressons-nous aux rayonnages sur lesquels on entrepose les différents produits.

Q5.1. Écrivez une classe `Shelf`, représentant un rayonnage sur lequel on entreposera des produits. Chaque étagère est caractérisée par le fait qu'elle soit réfrigérée ou non, par sa capacité d'accueil maximale, et par la liste des produits stockés dessus (de type `ArrayList`). Les spécifications à respecter sont les suivantes :



Q5.2. Ajoutez les méthodes `getShelves()`, `isFull()`, `isEmpty()` et `isRefrigerated()` retournant le contenu et déterminant respectivement si l'étagère est pleine, vide ou si elle est réfrigérée.

```

ArrayList<IProduct> getShelves()
boolean isFull()
boolean isEmpty()
boolean isRefrigerated()

```

Q5.3. Ajoutez une méthode `toString` retournant une chaîne de caractères de la forme : `[false : 1 -> []]` pour une étagère non réfrigérée, où l'on peut poser au maximum un produit, mais qui est actuellement vide.

Q5.4. Munissez cette classe d'une méthode `add` afin d'ajouter un produit donné en paramètre au rayonnage, à condition qu'ils soient compatibles, et sous réserve qu'il reste encore de la place. La méthode doit retourner `true` si l'ajout a pu être réalisé, `false` le cas échéant.

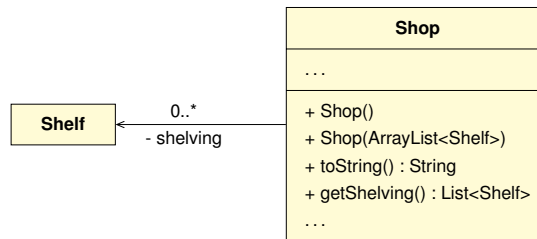
```

boolean add(IProduct p)

```

Q6. Intéressons-nous au magasin.

Q6.1. Créez une classe `Shop`, représentant une boutique, mais restreinte à un ensemble de rayonnages (de type `ArrayList`).



Q7. Écrivez une méthode `tidy`, prenant en paramètre un stock de produits et rangeant les produits sur les différentes étagères.

```
void tidy(ArrayList<IProduct> aStock)
```

Q7.1. On souhaite pouvoir lister et afficher tous les produits périssables du magasin dont la date limite de consommation approche (à `nbDays` près). Écrivez le code nécessaire pour réaliser cette tâche (à répartir dans les différentes classes).

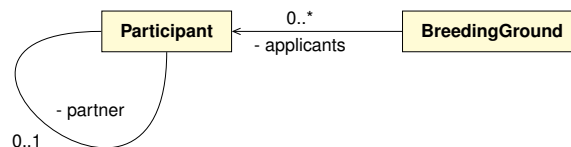
```
ArrayList<IProduct> closeBestBeforeDate(int nbDays)
```

Q8. Récupérez la classe de tests `ShopTest` et validez le fonctionnement de votre code.

Q9. Réalisez un `commit` associé au message : `"tp00-08::exo-miseEnRayon"`. Soumettez ensuite l'ensemble des `commit` au dépôt.

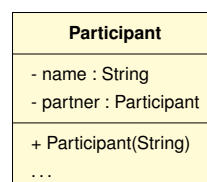
Exercice 2 : Concours et viviers de participants

On souhaite gérer l'inscription de participants à un concours. Seules des paires de participants peuvent participer à ce concours, mais rien n'empêche un participant de s'inscrire seul. On pourra lui trouver un partenaire dans le reste du vivier de participants le cas échéant. En revanche, chaque participant ne peut s'inscrire qu'une seule fois au concours. Impossible d'y participer plusieurs fois, même avec de multiples partenaires. La modélisation suivante est utilisée :



Q1. Commençons par les participants.

Q1.1. Écrivez une classe `Participant` répondant aux spécifications détaillées suivantes, issues du schéma UML précédent :



Q1.2. Ajoutez les accesseurs pour les deux attributs, ainsi qu'une méthode `getPartnerName()`, retournant le nom du partenaire impliqué.

```
String getPartnerName()
```

Q1.3. Adjoignez une méthode retournant un participant sous la forme : `"[<nom> -> <partenaire>]"`.

```
String toString()
```

Q1.4. Écrivez deux méthodes `isMatched()` et `isMatchedWith(...)`, qui retournent respectivement `true` si le participant est associé avec quelqu'un (peu importe qui) ou avec un autre participant donné, et `false` sinon.

```
boolean isMatched()
boolean isMatchedWith(Participant p)
```

Q1.5. Écrivez les méthodes `matchWith(...)` qui associe le participant courant au participant passé en paramètre, et `breakOff()` qui les séparent. Attention à la réflexivité ! La méthode `matchWith(...)` renvoie `true` si l'association entre le participant courant et celui passé en paramètre est possible, et `false` si l'un des deux a déjà un partenaire.

```
boolean matchWith(Participant p)
void breakOff()
```

Q2. À l'aide de votre *IDE*, générez les méthodes `hashCode()` et `equals(...)` qui serviront par la suite. Veillez à vous baser sur les "*bons*" attributs.

```
int hashCode()
boolean equals(Object obj)
```

Q3. Considérons maintenant le vivier de participants.

Q3.1. Écrivez la classe `BreedingGround` dépourvue de constructeur, mais munie d'un accesseur. Puisque les participants ne peuvent s'inscrire plusieurs fois, ils seront regroupés dans un `HashSet`.

Q3.2. Écrivez une méthode `registration(...)` qui permet à un participant de s'inscrire à un vivier, seulement et seulement s'il n'en n'est pas déjà membre.

```
boolean registration(Participant p)
```

Q4. Certains participants se sont inscrits seuls aux viviers du concours, mais la participation doit obligatoirement être par paires. Il faut donc être en mesure d'identifier tous les candidats isolés et forcer la création de nouvelles paires.

Q4.1. Écrivez la méthode `loners()` retournant la liste des participants du vivier qui n'ont pas de partenaires.

```
List<Participant> loners()
```

Q4.2. Écrivez la méthode `lonersCleansing()` qui ôte du vivier tous les participants qui n'ont pas de partenaire.

```
List<Participant> lonersCleansing()
```

Q5. On veut fournir un partenaire à tous les participants qui n'en n'ont pas. Écrivez la méthode `forcedMatching()` qui va sélectionner aléatoirement un partenaire à tous ceux qui n'en n'ont pas (si possible).

```
void forcedMatching()
```

Q6. Pour des raisons logistiques et organisationnelles, la sélection des candidats se fait de manière régionale. Or, certains essaient de s'inscrire à partir de différentes régions afin de tenter leur chance plusieurs fois. Il faut donc faire la chasse aux tricheurs.

Q6.1. Écrivez une méthode `cheaters(...)` qui détermine la liste des tricheurs (ceux étant inscrit dans le vivier courant et celui passé en paramètre).

```
List<Participant> cheaters(BreedingGround anotherBreedingGround)
```

Q6.2. Avant d'exclure les tricheurs, il faut s'assurer qu'ils ne sont plus associés à d'autres participants. Écrivez la méthode `isolateCheaters(...)` qui isole un tricheur donné. Ajoutez ensuite la méthode `cheatersCleansing()` qui exclue tous les tricheurs entre le vivier courant et celui passé en paramètre.

```
void isolateCheaters(Participant cheater)
void cheatersCleansing(BreedingGround anotherBreedingGround)
```

Q7. Une fois les différents viviers de candidats établis, on souhaite les regrouper si c'est possible.

Q7.1. Écrivez la méthode `possibleMerging(...)` qui détermine si la fusion est possible, c'est-à-dire s'il n'y a pas de tricheur (participant en commun aux deux viviers).

```
boolean possibleMerging(BreedingGround anotherBreedingGround)
```

Q7.2. Écrivez la méthode `merge(...)` qui effectue la fusion entre le vivier courant et le vivier passé en paramètre.

```
void fusion(BreedingGround anotherBreedingGround)
```

Q7.3. Écrivez la méthode `securedMerging`, qui teste si la fusion est possible et l'effectue. Cette méthode doit rendre la fusion possible le cas échéant (en bannissant les tricheurs).

```
void securedMerging(BreedingGround anotherBreedingGround)
```

Q8. Récupérez la classe de tests `BreedingGroundTest` et validez le fonctionnement de votre code.

Q9. Réalisez un `commit` associé au message : `"tp00-08::exo-concours"`. Soumettez ensuite l'ensemble des `commit` au dépôt.